

Redis

Database Notes — Knowledge Base

1. Overview

Redis (Remote Dictionary Server) is an open-source, in-memory data structure store used as a database, cache, message broker, and streaming engine. Because data lives primarily in RAM, Redis delivers sub-millisecond latency.

2. Supported Data Structures

- Strings — simple key-value pairs, also used for counters (INCR/DECR).
- Hashes — field-value maps, good for representing objects.
- Lists — ordered collections, usable as queues (LPUSH/RPOP).
- Sets and Sorted Sets — unique elements; sorted sets are ideal for leaderboards and ranking.
- Streams — append-only log structure for event/message processing.

3. Common Use Cases

- Caching — reducing load on primary databases by caching frequent query results.
- Session storage for web applications.
- Rate limiting and counters.
- Pub/Sub messaging between services.
- Real-time leaderboards using sorted sets.
- Distributed locks (e.g., via the Redlock algorithm).

4. Persistence Options

Although primarily in-memory, Redis supports optional persistence: RDB (point-in-time snapshots) and AOF (append-only file logging every write). These can be combined for durability, though Redis is still best treated as a fast cache/complementary store rather than a system of record for critical data.

5. Example Commands

```
SET user:1:name "Alice"  
EXPIRE user:1:name 3600  
ZADD leaderboard 1500 "player1"  
ZRANGE leaderboard 0 -1 WITHSCORES  
LPUSH task_queue "job123"
```

6. Trade-offs

- Data size limited by available RAM (though clustering helps distribute this).

- Not designed as the primary source of truth for complex relational data.
- Requires careful eviction policy configuration (e.g., LRU) when memory fills up.